

SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 1- EXPERIMENTS

Course Code:	CSA1223	Course Title:	Computer Architecture
CO Assessed:	CO3	Assessment:	ASSESSMENT TOOL 1- EXPERIMENTS
Weightage:	40%	Total Marks:	100
CO3 Statement:	<i>Design processor organization by constructing pipelined datapath structures, identifying hazard types (data, control, structural), and comparing superscalar techniques such as out-of-order execution, speculative execution, and simultaneous multithreading (SMT). (BL6)</i>		
Student Name:	M. Nivedha	Reg.No.:	192471040
Department:	CS & BS	Date:	19/8/2026

ANNEXURE A

1. Design and simulate a system that performs register transfer operations along with basic arithmetic and logic operations such as addition, subtraction, AND, and OR. Use multiple registers to demonstrate how data is transferred between them during execution, and analyze the sequence of operations and the role of the ALU in processing data.
2. Create a model of a computer system using single-bus and multi-bus organization, and perform data transfer operations between registers, memory, and I/O. Compare the time taken and efficiency of both approaches, and analyze how multiple bus structures improve parallel data transfer and reduce bottlenecks.
3. Simulate a 5-stage instruction pipeline (Fetch, Decode, Execute, Memory, Write-back) and execute a set of instructions. Measure the performance in terms of speedup and throughput, and compare it with a non-pipelined system to evaluate the effectiveness of pipelining.
4. Develop a pipeline execution scenario where data hazards, control hazards, and structural hazards occur. Observe their impact on instruction execution, and implement techniques such as stalling, data forwarding, and branch prediction to resolve these hazards and improve performance.
5. Analyze a processor supporting superscalar execution, including out-of-order and speculative execution, and extend the study to include hyper-threading and simultaneous multithreading (SMT). Evaluate how multiple instructions are executed in parallel and compare the performance improvements in terms of instruction throughput and resource utilization.

Code of Conduct:

I M. Nivedha (192471040) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

M. Nivedha
Signature of the student:

MARKS ALLOCATION

	Understanding of Concepts (20%)	Experimental Design (20%)	Use of Tools (25%)	Analysis of Results (20%)	Documentation (15%)
Q1	5	5	4	3	2
Q2	4	4	4	2	2
Q3	5	6	3	3	1.5
Q4	4	5	3	3	2
Q5	5	5	4	2	1

19
16
18.5
17
17

RUBRICS FOR CALCULATION:

87.5

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies (5)	Good understanding with proper integration of most concepts (4)	Basic understanding; limited or partial integration (3)	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution (6)	Correct implementation with minor issues (4)	Basic implementation with errors or inefficiencies (3)	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration (4)	Appropriate use of tools with correct execution (3)	Limited or inconsistent use of tools (2)	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results (3)	Good analysis with reasonable interpretation (2)	Basic analysis with limited insights (1)	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results (2)	Organized documentation with necessary details	Basic documentation with missing details (1)	Poor or incomplete documentation

SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 1- EXPERIMENTS

Course Code:	CSA1223	Course Title:	Computer Architecture
CO Assessed:	CO3	Assessment:	ASSESSMENT TOOL 1- EXPERIMENTS
Weightage:	40%	Total Marks:	100
CO3 Statement:	<i>Design processor organization by constructing pipelined datapath structures, identifying hazard types (data, control, structural), and comparing superscalar techniques such as out-of-order execution, speculative execution, and simultaneous multithreading (SMT). (BL6)</i>		
Student Name:	M.NIVEDHA	Reg.No.:	192471040
Department:	CS&BS	Date:	19.8.2026

ANNEXURE A

1. **Design and simulate a system that performs register transfer operations along with basic arithmetic and logic operations such as addition, subtraction, AND, and OR. Use multiple registers to demonstrate how data is transferred between them during execution, and analyze the sequence of operations and the role of the ALU in processing data.**

Register Transfer and ALU Operations

1. Aim

To design and simulate a system that performs register transfer operations and basic arithmetic and logic operations such as **Addition, Subtraction, AND, and OR using an ALU.**

2. Objectives

- To understand data transfer between registers.
- To perform arithmetic operations using the ALU.
- To perform logical operations using the ALU.
- To understand the role of the control unit in processor datapath operations.

3. Software / Tools Required

Software: Logisim Evolution / Digital Logic Simulator

Components:

- Registers R1, R2 and R3
- ALU

- Multiplexer
- Control unit
- Clock
- Input switches
- LEDs/Display
- Wires

4. Theory

A **register transfer operation** is the movement of data from one register to another under the control of control signals.

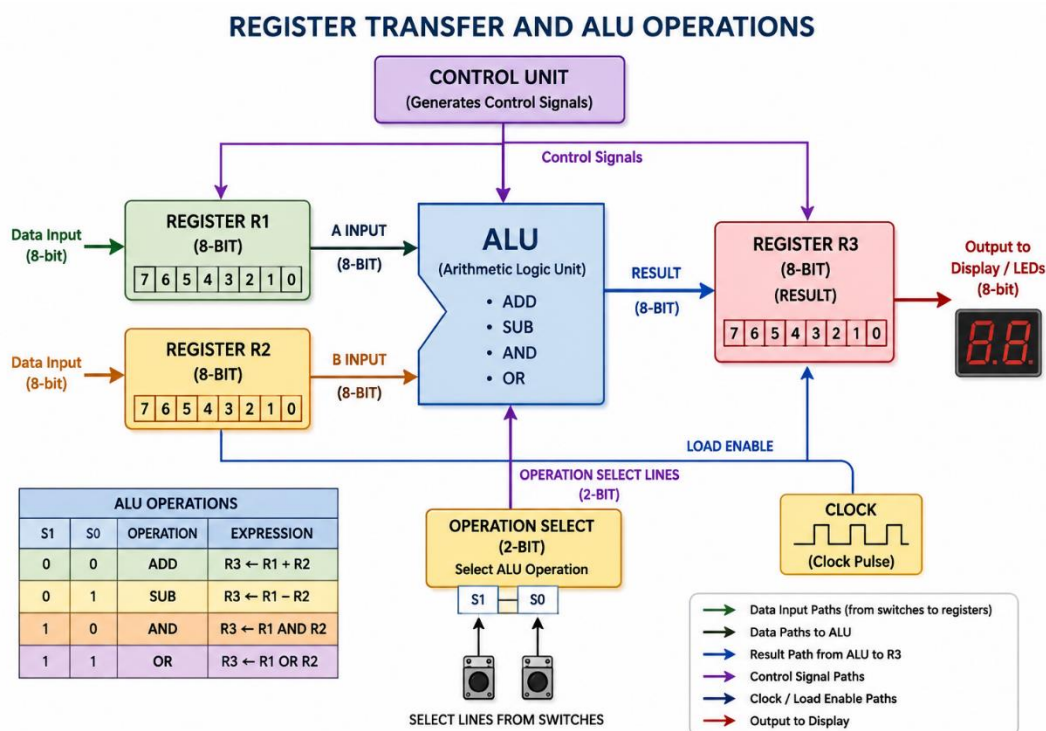
Examples:

- $R3 \leftarrow R1$
- $R3 \leftarrow R1 + R2$
- $R3 \leftarrow R1 - R2$
- $R3 \leftarrow R1 \text{ AND } R2$
- $R3 \leftarrow R1 \text{ OR } R2$

The **ALU (Arithmetic Logic Unit)** performs arithmetic and logical operations.

The control unit generates control signals that select the required ALU operation and control data movement.

5. Block Diagram



6. ALU Operations

Control	Operation	Expression
00	ADD	$R3 \leftarrow R1 + R2$
01	SUB	$R3 \leftarrow R1 - R2$
10	AND	$R3 \leftarrow R1 \text{ AND } R2$
11	OR	$R3 \leftarrow R1 \text{ OR } R2$

7. Procedure

1. Open **Logisim Evolution**.
2. Place three registers and name them **R1, R2 and R3**.
3. Place an **ALU** and connect R1 and R2 to its inputs.
4. Connect the ALU output to R3.
5. Connect a control unit or switches to the ALU operation-selection inputs.
6. Connect a clock to the registers.
7. Enter values into R1 and R2.
8. Select the required ALU operation.
9. Apply the clock pulse.
10. Observe the output stored in R3.
11. Repeat the experiment for ADD, SUB, AND and OR.

8. Sample Input

Let:

$$R1 = 10 = 1010_2$$

$$R2 = 5 = 0101_2$$

9. Observation Table

Operation	R1	R2	ALU Output	Decimal
Addition	1010	010	1111	15
Subtraction	1010	010	0101	5
AND	1010	010	0000	0

Operation	R1	R2	ALU Output	Decimal
OR	1010	0101	1111	15

10. Register Transfer Sequence

Step 1: $R1 \leftarrow 10$

Step 2: $R2 \leftarrow 5$

Step 3: ALU selects ADD

Step 4: $R3 \leftarrow R1 + R2$

Step 5: $R3 = 15$

For subtraction:

$R3 \leftarrow R1 - R2$

$R3 = 10 - 5$

$R3 = 5$

11. Analysis

The experiment demonstrates that registers temporarily store operands and results, while the ALU performs the required computation. The control signals determine which operation is selected.

For the given inputs:

- Addition produces **15**.
- Subtraction produces **5**.
- AND produces **0**.
- OR produces **15**.

Thus, the ALU successfully performs both arithmetic and logical operations.

12. Result

The register transfer system and ALU were successfully designed and simulated. The system correctly performed **addition, subtraction, AND and OR operations** and transferred the results between registers.

13. Conclusion

The experiment demonstrates the interaction between **registers, ALU and control unit** in a processor datapath. Register transfers provide the movement of data, while the ALU performs the required arithmetic and logical processing.

2. Create a model of a computer system using single-bus and multi-bus organization, and perform data transfer operations between registers, memory, and I/O. Compare the time taken and efficiency of both approaches, and analyze how multiple bus structures improve parallel data transfer and reduce bottlenecks.

Single-Bus and Multi-Bus Organization

1. Aim

To design and simulate **single-bus and multi-bus computer organizations** and perform data transfer operations between registers, memory and I/O. To compare their **execution time, efficiency and parallel data-transfer capability**.

2. Objectives

- To understand single-bus and multi-bus organization.
- To perform register, memory and I/O data transfers.
- To compare the performance of both architectures.
- To analyze how multiple buses reduce data-transfer bottlenecks.

3. Software / Tools Required

Software: Logisim Evolution / Digital Logic Simulator

Components:

- Registers R1, R2, R3, R4
- Memory
- I/O module
- ALU
- Multiplexers
- Bus lines
- Control unit
- Clock
- LEDs/Display

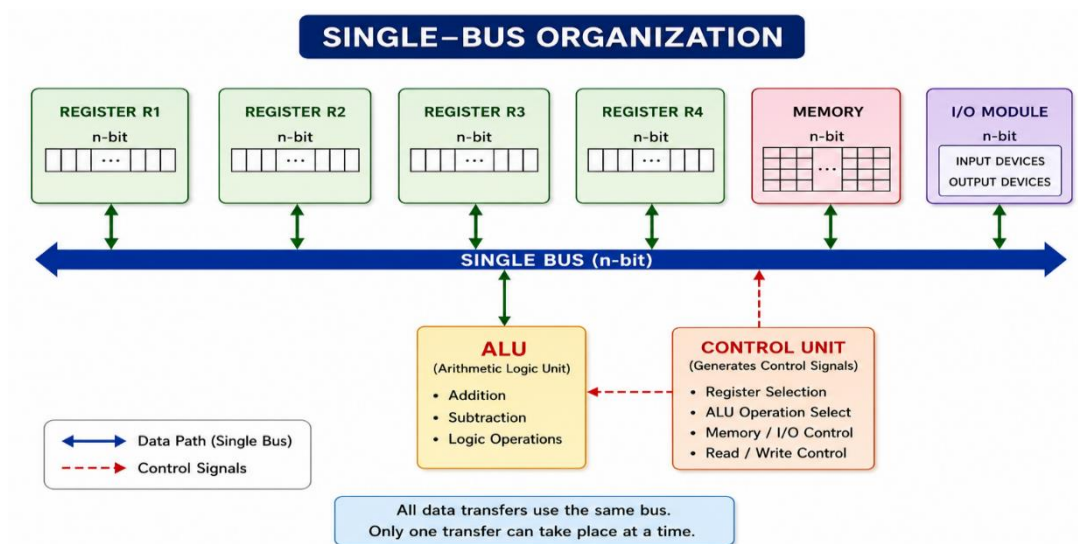
4. Theory

A **bus** is a common communication path used to transfer data between processor components.

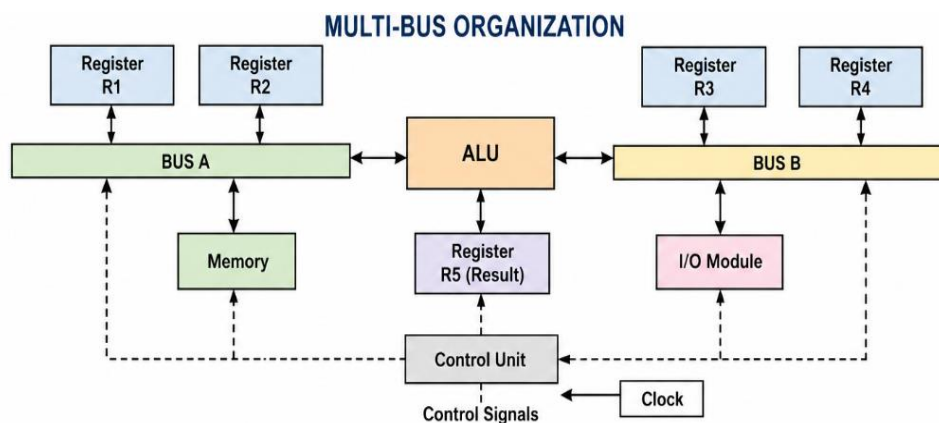
In a **single-bus organization**, registers, memory, ALU and I/O share one common bus. Since only one transfer can normally use the bus at a time, it may create a bottleneck.

In a **multi-bus organization**, two or more buses are available. This allows multiple data transfers to take place simultaneously, improving parallelism and reducing execution time.

5. Single-Bus Block Diagram



6. Multi-Bus Block Diagram



7. Data Transfer Operations

Examples:

R1 → R3

R2 → R4

Memory → R1

R2 → Memory

I/O → R1

In a multi-bus system, independent transfers can occur in parallel when the architecture and control logic permit them.

8. Procedure

1. Open **Logisim Evolution**.
2. Construct the single-bus architecture.
3. Connect registers, memory and I/O to the common bus.
4. Connect the ALU to the bus.
5. Perform register-to-register data transfer.
6. Perform memory-to-register and register-to-memory transfers.
7. Record the number of clock cycles.
8. Construct a multi-bus architecture using two or more buses.
9. Connect registers and ALU to the separate buses.
10. Perform the same data-transfer operations.
11. Record the execution cycles.
12. Compare both architectures.

9. Observation Table

Operation	Single Bus	Multi Bus
$R1 \rightarrow R3$	1 cycle	1 cycle
$R2 \rightarrow R4$	1 cycle	1 cycle
$R1 + R2 \rightarrow R3$	3 cycles*	2 cycles*
Memory $\rightarrow R1$	2 cycles*	1–2 cycles*
Parallel transfers	Limited	Possible

Example values depend on the particular simulator design and control implementation.

10. Performance Comparison

Parameter	Single-Bus	Multi-Bus
Number of buses	1	2 or more
Parallel transfers	Low	High
Bus contention	High	Lower
Data-transfer speed	Lower	Higher
Hardware complexity	Low	High
Cost	Low	Higher

Parameter	Single-Bus	Multi-Bus
Bottleneck	More	Less
Efficiency	Moderate	Higher

11. Analysis

The single-bus architecture is simple because all components use one common communication path. However, multiple components competing for the same bus can cause delays.

The multi-bus architecture provides separate paths for data transfer. Therefore, operands can reach the ALU more efficiently and independent transfers can occur simultaneously.

For example:

Single Bus:

$R1 \rightarrow \text{BUS} \rightarrow \text{ALU}$

$R2 \rightarrow \text{BUS} \rightarrow \text{ALU}$

Transfers must share the same path.

In a multi-bus architecture:

$R1 \rightarrow \text{BUS A} \text{ — } \text{┐}$
 $\text{┌ — } \text{▶ ALU}$

$R2 \rightarrow \text{BUS B} \text{ — } \text{┐}$

Both operands can be supplied to the ALU simultaneously.

12. Result

The **single-bus and multi-bus organizations** were successfully designed and simulated. The multi-bus architecture provided better parallel data-transfer capability and reduced bus bottlenecks.

13. Conclusion

The experiment shows that a single-bus architecture is simpler and less costly, whereas a multi-bus architecture provides higher performance and better parallelism by allowing multiple data paths. Single-bus systems are suitable for simple and low-cost processor designs. Multi-bus systems reduce data transfer bottlenecks and improve CPU efficiency. Multiple buses allow simultaneous transfer of data between different processor components. Therefore, the choice between single-bus and multi-bus architecture depends on the required performance, complexity, and cost.

3. **Simulate a 5-stage instruction pipeline (Fetch, Decode, Execute, Memory, Write-back) and execute a set of instructions. Measure the performance in terms of speedup and throughput, and compare it with a non-pipelined system to evaluate the effectiveness of pipelining.**

Five-Stage Instruction Pipeline

1. Aim

To design and simulate a **5-stage instruction pipeline** consisting of Fetch, Decode, Execute, Memory and Write-back stages, and compare its performance with a **non-pipelined processor** using speedup and throughput.

2. Objectives

- To understand the concept of instruction pipelining.
- To simulate the five pipeline stages.
- To observe overlapping execution of instructions.
- To calculate execution time, speedup and throughput.
- To compare pipelined and non-pipelined execution.

3. Software / Tools Required

- **Python / Logisim Evolution**
- Computer system
- Pipeline simulation model
- Instruction set
- Clock cycle counter

4. Theory

Instruction pipelining divides instruction execution into multiple stages. Each stage performs a part of the instruction operation.

The standard five stages are:

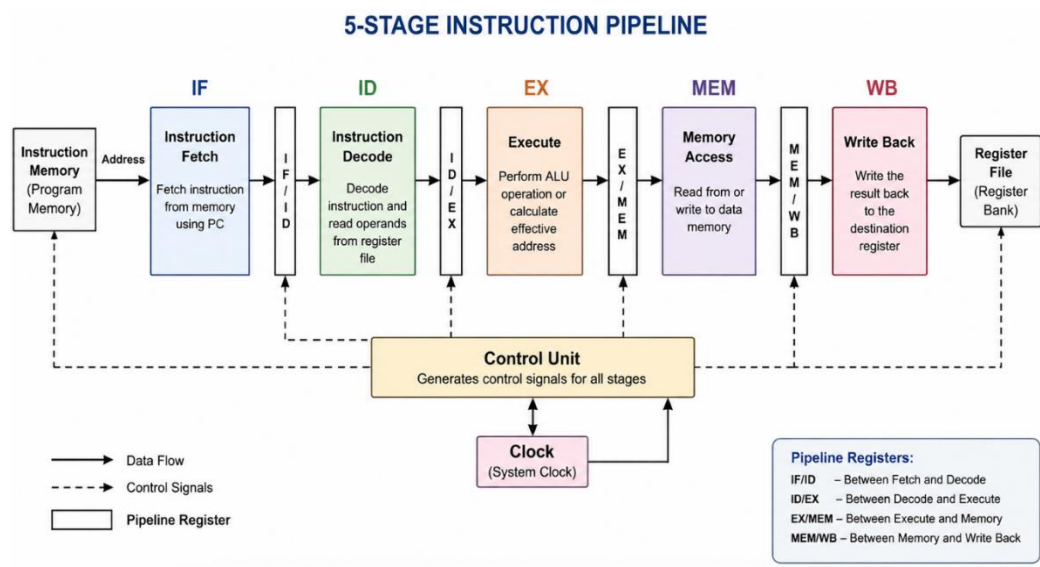
1. **IF – Instruction Fetch:** Fetches the instruction from memory.
2. **ID – Instruction Decode:** Decodes the instruction and reads operands.
3. **EX – Execute:** Performs ALU operation.
4. **MEM – Memory:** Performs memory read/write if required.
5. **WB – Write Back:** Stores the result into the destination register.

The major advantage is **overlapping execution**. While one instruction is being executed, another can be decoded and another can be fetched.

Key Points

- Multiple instructions can execute simultaneously in different stages.
- After the pipeline is filled, ideally one instruction can complete in every clock cycle.
- Pipeline registers store intermediate results between stages.
- Pipelining improves instruction throughput and CPU resource utilization.
- Performance can be affected by data, control and structural hazards.

5. Block Diagram



6. Pipeline Stages

Stage	Name	Function
IF	Instruction Fetch	Fetch instruction
ID	Instruction Decode	Decode and read operands
EX	Execute	Perform ALU operation
MEM	Memory Access	Access memory
WB	Write Back	Store result

7. Example Instruction Set

Consider five instructions:

I1 : ADD R1, R2, R3

I2 : SUB R4, R5, R6

I3 : AND R7, R8, R9

I4 : OR R10, R11, R12

I5 : ADD R13, R14, R15

Assume each pipeline stage takes **one clock cycle**.

8. Pipeline Execution

	Clock Cycle									
Instruction	1	2	3	4	5	6	7	8	9	

I1		IF	ID	EX	MEM	WB				
I2			IF	ID	EX	MEM	WB			
I3				IF	ID	EX	MEM	WB		
I4					IF	ID	EX	MEM	WB	
I5						IF	ID	EX	MEM	WB

The pipeline becomes fully occupied after the initial stages are filled.

9. Non-Pipelined Execution

Without pipelining, one instruction completes all five stages before the next instruction starts.

For **5 instructions**:

Number of cycles = Instructions $\times$ Stages

= 5×5

= 25 cycles

Non-Pipelined Schedule

I1 $\rightarrow$ IF $\rightarrow$ ID $\rightarrow$ EX $\rightarrow$ MEM $\rightarrow$ WB

I2 $\rightarrow$ IF $\rightarrow$ ID $\rightarrow$ EX $\rightarrow$ MEM $\rightarrow$ WB

I3 $\rightarrow$ IF $\rightarrow$ ID $\rightarrow$ EX $\rightarrow$ MEM $\rightarrow$ WB

I4 $\rightarrow$ IF $\rightarrow$ ID $\rightarrow$ EX $\rightarrow$ MEM $\rightarrow$ WB

I5 $\rightarrow$ IF $\rightarrow$ ID $\rightarrow$ EX $\rightarrow$ MEM $\rightarrow$ WB

Total = 25 cycles

10. Pipelined Execution

For an ideal pipeline:

Pipeline cycles = $k + (n - 1)$

Where:

- k = number of stages = 5

- $n = \text{number of instructions} = 5$

Therefore:

$$= 5 + (5 - 1)$$

$$= 9 \text{ cycles}$$

11. Speedup Calculation

Formula

Speedup = Non-pipelined execution time / Pipelined execution time

Therefore:

$$\text{Speedup} = 25 / 9$$

$$\text{Speedup} \approx 2.78$$

So, the idealized example achieves approximately **2.78× speedup**.

12. Throughput

Throughput indicates the number of instructions completed per unit time.

For the simplified model, after the pipeline is filled, approximately **one instruction completes every clock cycle**.

If the clock period is T :

$$\text{Throughput} \approx 1/T \text{ instructions per cycle time}$$

For 5 instructions in 9 cycles:

$$\text{Average throughput} = 5/9 \approx 0.556 \text{ instructions/cycle}$$

The steady-state throughput approaches **1 instruction/cycle** for a long instruction stream, assuming no hazards or stalls.

13. Observation Table

Observation Table

Parameter	Non-Pipelined	Pipelined
Instructions	5	5
Stages	5	5
Total Cycles	25	9
Speedup	1×	2.78×
Instruction Overlap	No	Yes

Parameter	Non-Pipelined	Pipelined
Throughput	Low	High

14. Experimental Procedure

1. Open the pipeline simulation environment.
2. Create the five stages: **IF, ID, EX, MEM and WB**.
3. Enter the five sample instructions.
4. Assign one clock cycle to each stage.
5. Start instruction I1.
6. In the next clock cycle, move I1 to ID and fetch I2.
7. Continue moving instructions through the pipeline.
8. Record the clock cycle for each stage.
9. Calculate total pipelined execution time.
10. Calculate the execution time without pipelining.
11. Calculate speedup.
12. Compare throughput and performance.

15. Analysis

The non-pipelined processor requires **25 cycles** because every instruction must complete all five stages before the next instruction begins.

The pipelined processor requires only **9 cycles** for the same five instructions because different instructions are processed simultaneously in different stages.

Thus: **25 cycles → 9 cycles**

This demonstrates that pipelining significantly improves instruction throughput.

The calculated speedup is: **2.78×**

In a real processor, the actual speedup may be lower because of data hazards, control hazards, structural hazards, pipeline stalls and unequal stage delays.

16. Result

The **5-stage pipeline** was successfully simulated. It completed 5 instructions in **9 cycles**, achieving an approximate **2.78× speedup** over non-pipelined execution.

17. Conclusion

Instruction pipelining improves processor performance by overlapping instruction execution. It increases throughput and resource utilization compared with non-pipelined execution.

4. **Develop a pipeline execution scenario where data hazards, control hazards, and structural hazards occur. Observe their impact on instruction execution, and implement techniques such as stalling, data forwarding, and branch prediction to resolve these hazards and improve performance.**

4. Pipeline Hazards and Hazard Resolution

1. Aim

To design and simulate a pipeline execution scenario containing data, control and structural hazards, observe their impact on instruction execution, and implement stalling, data forwarding and branch prediction techniques to improve performance.

2. Objectives

- To identify different types of pipeline hazards.
- To study the effect of hazards on pipeline performance.
- To implement stalling for hazard resolution.
- To implement data forwarding for data hazards.
- To use branch prediction for control hazards.
- To analyze performance before and after hazard resolution.

3. Software / Tools Required

- **Python / Logisim Evolution**
- Computer system
- Pipeline simulator
- Registers
- ALU
- Memory
- Branch unit
- Clock

6.Theory

A pipeline hazard is a condition that prevents the next instruction from executing in its normal clock cycle.

It may cause delays, stalls, or incorrect execution in the pipeline

There are three major types:

A. Data Hazard

Occurs when an instruction depends on the result of an earlier instruction.

I1: ADD R1, R2, R3

I2: SUB R4, R1, R5



Data Dependency

I2 needs the value of R1 produced by I1.

B. Control Hazard

Occurs due to branch or jump instructions.

BEQ R1, R2, LABEL



Branch Decision

The processor may fetch the wrong instruction before knowing whether the branch is taken.

C. Structural Hazard

Occurs when two instructions require the same hardware resource at the same time.

Example:

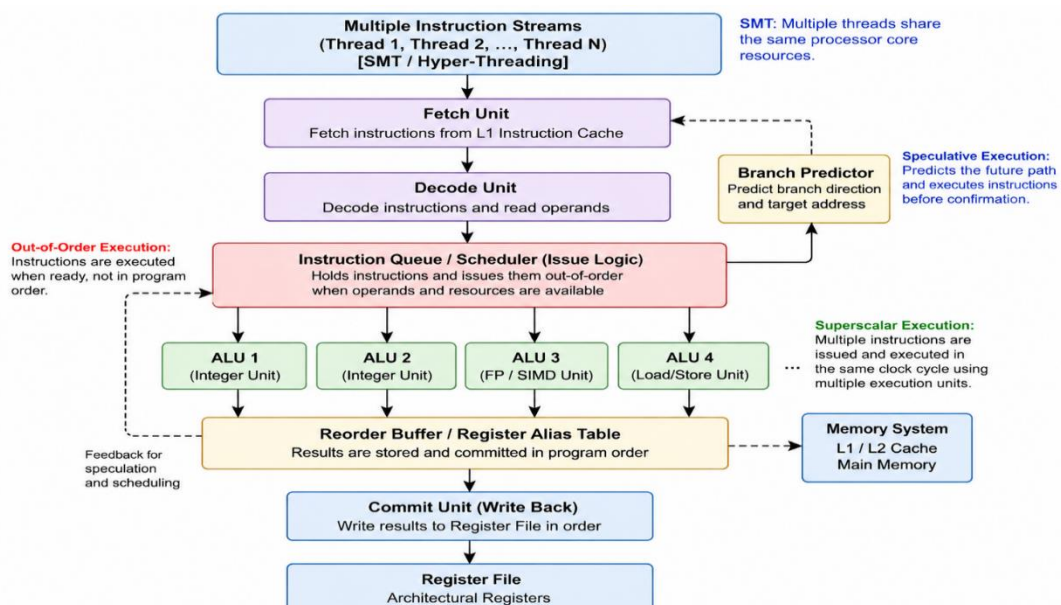
Instruction 1 —► Memory

Instruction 2 —► Memory



Same resource

5. Block Diagram



6. Data Hazard Example

Consider:

I1: ADD R1, R2, R3

I2: SUB R4, R1, R5

I2 requires the result of I1.

Without Forwarding

Cycle 1 2 3 4 5 6

I1 IF ID EX MEM WB

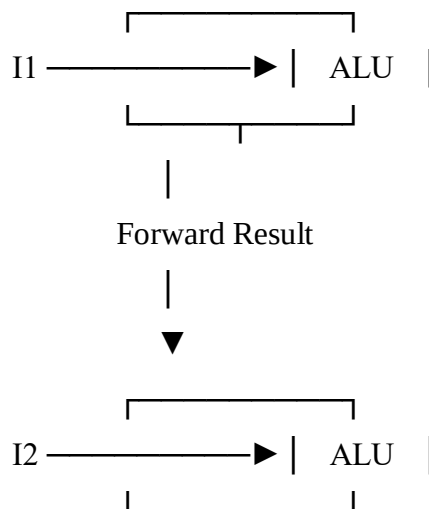
I2 IF ID ST EX MEM

ST = Stall

The second instruction must wait for the required data.

7. Data Forwarding

Instead of waiting until Write Back, the result can be forwarded directly from a pipeline stage.



This reduces unnecessary stalls.

8. Control Hazard Example

Consider:

I1: BEQ R1, R2, LABEL

I2: ADD R3, R4, R5

I3: SUB R6, R7, R8

The processor may fetch I2 before knowing whether the branch is taken.

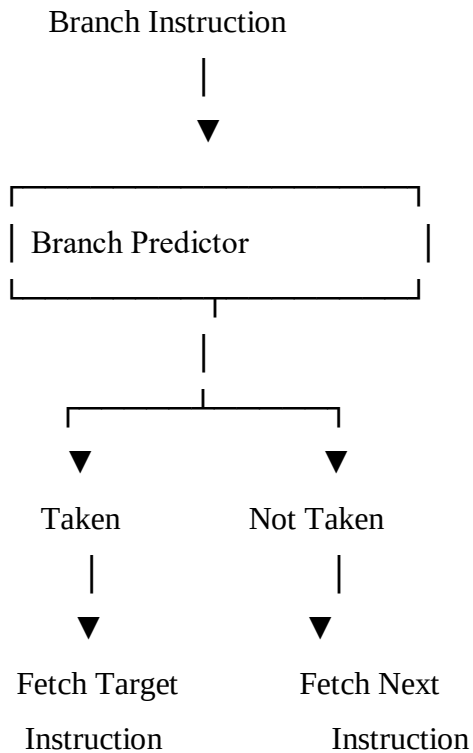
Key points:

- BEQ checks whether R1 and R2 are equal.

- If $R1 = R2$, the branch is **taken** and execution jumps to LABEL.
- If $R1 \neq R2$, the branch is **not taken** and execution continues with I2.
- The processor may fetch I2 before the branch decision is available.
- If the branch is taken, the already-fetched instructions may be incorrect and must be **flushed**.
- This causes **pipeline stalls and performance loss**.

Solution: Branch Prediction

The processor predicts whether the branch will be taken.

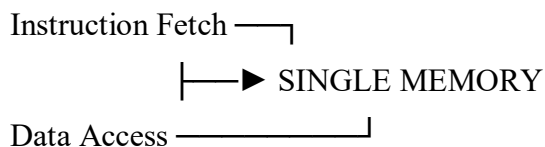


If the prediction is correct, execution continues with minimal delay.

If incorrect, incorrectly fetched instructions are discarded and the correct path is fetched.

9. Structural Hazard Example

Suppose the processor has a single memory used for both instruction fetching and data access.



If both operations occur during the same cycle, a conflict occurs.

Solutions

- Add separate instruction and data memories.
- Add additional hardware resources.
- Stall one instruction.
- Reschedule instructions.

10. Hazard Resolution Techniques

Hazard	Problem	Resolution
Data	Data dependency	Forwarding / Stalling
Control	Branch decision	Branch prediction / Stalling
Structural	Resource conflict	Resource duplication / Scheduling

11. Experimental Procedure

1. Create a **5-stage pipeline** consisting of IF, ID, EX, MEM and WB.
2. Load a set of instructions.
3. Introduce dependent instructions to create a data hazard.
4. Observe the pipeline stall.
5. Implement data forwarding.
6. Compare execution before and after forwarding.
7. Introduce a branch instruction.
8. Observe the control hazard.
9. Implement branch prediction.
10. Introduce a shared-resource conflict to create a structural hazard.
11. Resolve the structural hazard using an additional resource or scheduling.
12. Record the execution cycles.
13. Compare the performance before and after hazard resolution.

12. Observation Table

Hazard Type	Example	Effect	Technique
Data Hazard	ADD followed by dependent S	Waiting/Stall	Forwarding
Control Hazard	Branch instruction	Wrong-path fetch	Branch prediction
Structural Hazard	Two instructions use same resou	Resource conflict	Resource duplicatio

13. Performance Analysis

Consider an example where a pipeline requires **12 cycles without hazard handling**.

After applying forwarding, branch prediction and resource management, suppose the execution requires **9 cycles**.

Performance Improvement

$$\begin{aligned}\text{Improvement} &= 12 - 9 \\ &= 3 \text{ cycles}\end{aligned}$$

Percentage Improvement

$$\begin{aligned}\text{Improvement \%} &= (12 - 9) / 12 \times 100 \\ &= 25\%\end{aligned}$$

Thus, the example demonstrates a **25% reduction in execution cycles**.

The exact values in your simulator may differ depending on the instructions and pipeline configuration. Use your actual observed cycle counts in the final record if your faculty requires simulation-generated values.

14. Comparison

Parameter	Without Resolution	With Resolution
Data hazard	Stalls	Forwarding
Control hazard	Branch delay	Prediction
Structural hazard	Resource conflict	Additional resource
Execution cycles	Higher	Lower
Pipeline efficiency	Lower	Higher
Throughput	Lower	Higher

15. Analysis

Pipeline hazards reduce the performance improvement obtained from pipelining.

Data forwarding allows dependent instructions to receive results earlier without waiting for the normal Write-back stage.

Stalling is used when the required data or resource is not immediately available.

Branch prediction reduces the performance penalty caused by control hazards by predicting the branch direction.

Resource duplication can reduce structural hazards by allowing multiple instructions to access required resources simultaneously.

Therefore, hazard-resolution techniques improve pipeline utilization and reduce unnecessary delays.

16. Result

Data, control and structural hazards were successfully introduced and analyzed in the pipeline. Stalling, data forwarding and branch prediction were applied to reduce their effects and improve pipeline performance.

17. Conclusion

The experiment demonstrates that pipeline hazards can significantly reduce processor performance. Proper hazard-resolution techniques such as forwarding, stalling, branch prediction and resource duplication help maintain efficient pipeline execution and increase instruction throughput.

5. Analyze a processor supporting superscalar execution, including out-of-order and speculative execution, and extend the study to include hyper-threading and simultaneous multithreading (SMT). Evaluate how multiple instructions are executed in parallel and compare the performance improvements in terms of instruction throughput and resource utilization.

Superscalar, Out-of-Order, Speculative Execution and SMT

1. Aim

To analyze a processor supporting **superscalar execution, out-of-order execution, speculative execution, hyper-threading and simultaneous multithreading (SMT)**, and evaluate their effect on instruction throughput and resource utilization.

2. Objectives

- To understand superscalar processor architecture.
- To study parallel instruction execution.
- To understand out-of-order execution.
- To analyze speculative execution.
- To understand hyper-threading and SMT.
- To compare instruction throughput and resource utilization.

3. Software / Tools Required

- **Python / Logisim Evolution**
- Computer system
- Processor simulation model
- Multiple ALUs / execution units
- Registers
- Instruction scheduler

- Branch predictor
- Cache/memory model

4. Theory

Superscalar Execution

A **superscalar processor** can issue and execute multiple independent instructions in the same clock cycle using multiple execution units.

Example:

Cycle 1 → I1 + I2

Cycle 2 → I3 + I4

Instead of executing only one instruction per cycle, multiple instructions can execute simultaneously.

Out-of-Order Execution

In out-of-order execution, instructions are executed when their operands and required resources are available rather than strictly following program order.

Example:

I1 → Waiting for data

I2 → Ready → Execute

I3 → Ready → Execute

Therefore, I2 and I3 can execute while I1 is waiting.

Speculative Execution

Speculative execution predicts the future execution path, usually after a branch, and executes instructions before the branch outcome is confirmed.

- **Correct prediction** → results are retained.
- **Incorrect prediction** → speculative results are discarded.

SMT / Hyper-Threading

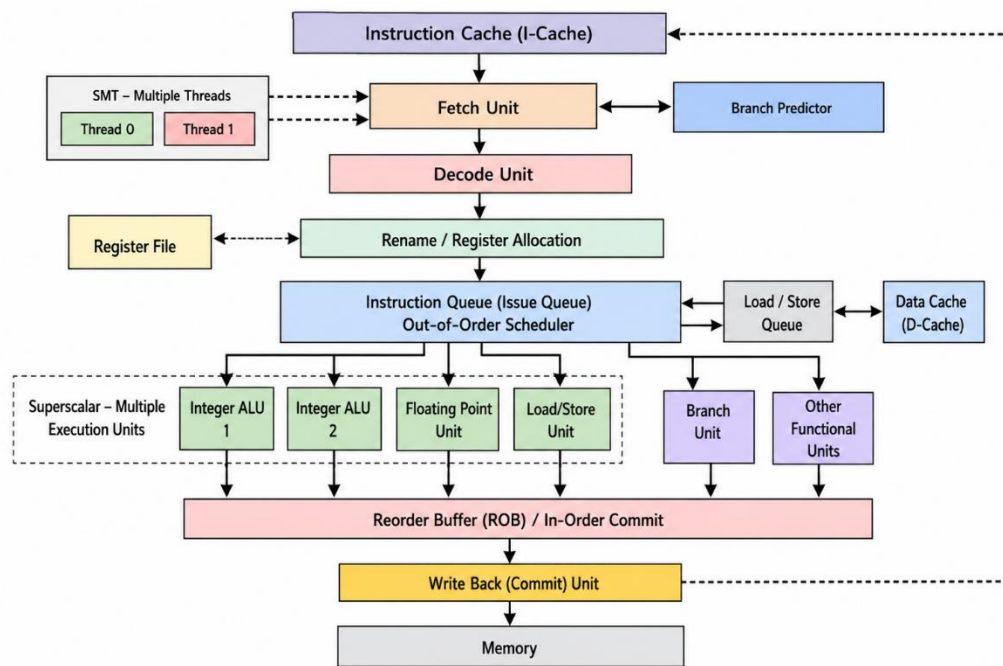
Simultaneous Multithreading (SMT) allows multiple threads to share the execution resources of one physical processor core.

Hyper-threading is Intel's implementation/branding of SMT in many processors.

- SMT allows instructions from different threads to execute simultaneously.
- It improves the utilization of CPU execution resources.
- When one thread is waiting for memory or another resource, another thread can use the available resources.
- SMT can increase overall instruction throughput.
- Multiple threads share resources such as execution units, caches and other processor resources.

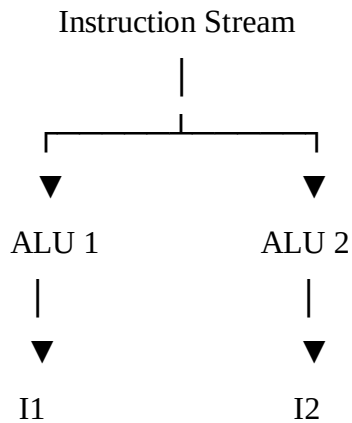
- SMT does not create a complete physical core for each thread.

5. Block Diagram



6. Superscalar Execution

Assume the processor has **two ALUs**.



If I1 and I2 are independent, both can execute in the same cycle.

Observation

Cycle	ALU 1	ALU 2
1	I1	I2
2	I3	I4

Cycle	ALU 1	ALU 2
3	I5	I6

Thus, up to **two instructions per cycle** can be issued under ideal conditions.

7. Out-of-Order Execution

Consider:

I1: LOAD R1, [1000]

I2: ADD R4, R5, R6

I3: SUB R7, R8, R9

I1 may take longer because it accesses memory.

I2 and I3 do not depend on I1, so they can execute first.

Program Order:

I1 → I2 → I3

Execution Order:

I2 → I3 → I1

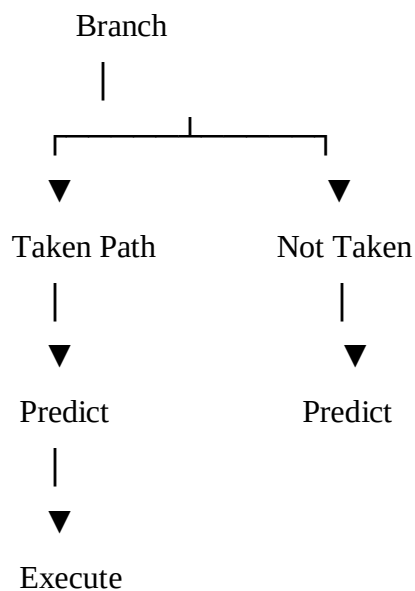
The processor maintains correct program-visible behavior by completing/committing results in the required architectural order.

8. Speculative Execution

Consider:

BEQ R1, R2, TARGET

The processor predicts whether the branch is taken.



Correct Prediction

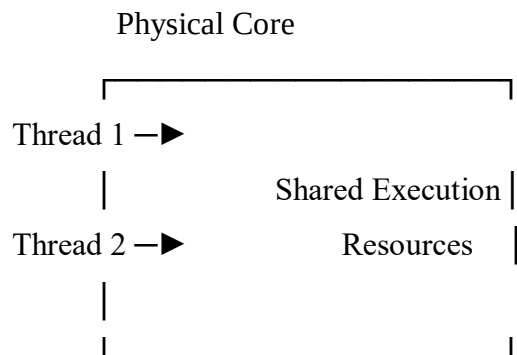
The processor continues execution without significant branch interruption.

Incorrect Prediction

The incorrectly executed instructions are discarded and the correct path is fetched.

9. SMT / Hyper-Threading

A physical processor core can execute instructions from multiple threads.



When one thread is waiting for memory or another resource, another thread can use available execution resources.

10. Experimental Procedure

1. Create a processor model with multiple execution units.
2. Load a sequence of independent instructions.
3. Execute multiple instructions in the same clock cycle.
4. Record the instruction throughput.
5. Introduce instructions with dependencies.
6. Apply out-of-order scheduling.
7. Introduce a branch instruction.
8. Implement branch prediction for speculative execution.
9. Create two or more instruction streams.
10. Implement an SMT model where the threads share processor resources.
11. Record execution cycles and resource utilization.
12. Compare the different execution techniques.

11. Observation Table

Technique	Instructions/Cycle	Main Benefit
Basic Pipeline	~1	Instruction overlap

Technique	Instructions/Cycle	Main Benefit
Superscalar	>1	Higher instruction throughput
Out-of-Order	Variable	Reduces waiting
Speculative Execution	Variable	Reduces branch delay
SMT	Multiple threads	Better resource utilization

12. Performance Example

Assume:

- 8 independent instructions
- 2 execution units
- Ideal conditions

Single-Issue Processor

Approximately:

$8 \text{ instructions} \div 1 \text{ instruction/cycle} = 8 \text{ cycles}$

2-Wide Superscalar Processor

Approximately:

$8 \text{ instructions} \div 2 \text{ instructions/cycle} = 4 \text{ cycles}$

Ideal Speedup

$\text{Speedup} = 8 / 4$

$= 2\times$

This is an idealized calculation. Actual performance depends on dependencies, branches, memory delays and available execution resources.

13. Resource Utilization

Consider a processor with two execution units.

Condition	ALU 1	ALU 2
Single instruction stream	Used	May be idle
Superscalar	Used	Used
Out-of-order	Used when ready	Used when ready
SMT	Thread 1	Thread 2 / shared scheduling

SMT can improve utilization when one thread cannot fully use all available resources.

14. Comparison

Technique	Simple Meaning	Main Benefit
Superscalar	Executes multiple instructions at the same time	Increases speed
Out-of-Order	Executes ready instructions first	Reduces waiting
Speculative Execution	Executes predicted instructions before confirmation	Reduces branch delays
SMT	Executes multiple threads on one core	Improves resource utilization
Hyper-Threading	Intel's implementation of SMT	Improves multitasking

15. Analysis

Superscalar execution increases performance by allowing multiple instructions to execute simultaneously.

Out-of-order execution prevents execution units from remaining idle when an earlier instruction is waiting for data.

Speculative execution reduces branch-related delays by predicting the likely execution path. However, incorrect predictions cause wasted work.

SMT allows multiple threads to share a physical core. This can improve resource utilization when one thread cannot fully utilize the available execution units.

Therefore, these techniques work together to exploit instruction-level and thread-level parallelism.

16. Result

The superscalar processor model was successfully analyzed with out-of-order execution, speculative execution and SMT. The study demonstrated that executing multiple instructions and threads in parallel can improve instruction throughput and processor resource utilization.

17. Conclusion

Superscalar execution, out-of-order execution, speculative execution and SMT are important techniques used in modern processors to improve performance. Superscalar execution increases instruction-level parallelism, out-of-order execution reduces waiting, speculative execution reduces branch delays, and SMT improves utilization of processor resources by supporting multiple threads.

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:**MARKS ALLOCATION**

	Understanding of Concepts (20%)	Experimental Design (20%)	Use of Tools (25%)	Analysis of Results (20%)	Documentation (15%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation